

(Additional)Exp 2: Comparative Analysis of Support Vector Machine (SVM) and Random Forest Classifiers Using the Iris Dataset

Aim

To implement and compare **Support Vector Machine (SVM)** and **Random Forest** classifiers on the Iris dataset using an alternative preprocessing method and evaluate their performance using **accuracy**, **classification report**, and **confusion matrix**.

Procedure

1. Import the necessary Python libraries.
2. Load the Iris dataset into a Pandas DataFrame.
3. Normalize the feature values using MinMaxScaler.
4. Split the dataset into training and testing sets.
5. Train the SVM classifier with the RBF kernel.
6. Predict the output for the test data.
7. Calculate the accuracy, confusion matrix, and classification report.
8. Train the Random Forest classifier.
9. Predict the output using the Random Forest model.
10. Compare the performance of both models using confusion matrix heatmaps.

CODE:

```
!pip install pandas numpy matplotlib seaborn scikit-learn

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["Species"] = iris.target

print("First Five Records")
print(df.head())

print("\nDataset Shape:", df.shape)

X = df.iloc[:, :-1]
```

```

y = df["Species"]

scaler = MinMaxScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=10,
    stratify=y
)

svm = SVC(kernel="rbf", gamma="scale")
svm.fit(X_train, y_train)

svm_pred = svm.predict(X_test)

print("\n===== SVM =====")
print("Accuracy :", accuracy_score(y_test, svm_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test, svm_pred))

print("\nClassification Report")
print(classification_report(y_test, svm_pred))

rf = RandomForestClassifier(
    n_estimators=150,
    criterion="gini",
    random_state=10
)

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)

print("\n===== RANDOM FOREST =====")
print("Accuracy :", accuracy_score(y_test, rf_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test, rf_pred))

print("\nClassification Report")
print(classification_report(y_test, rf_pred))

fig, ax = plt.subplots(1,2,figsize=(12,5))

sns.heatmap(
    confusion_matrix(y_test, svm_pred),
    annot=True,

```

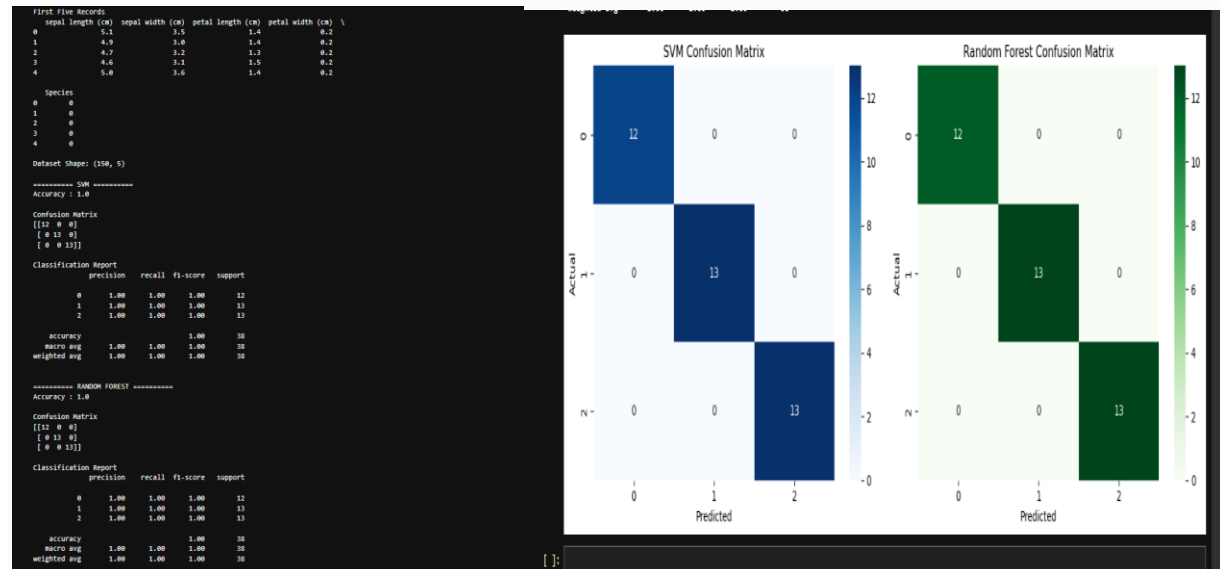
```

cmap="Blues",
fmt="d",
ax=ax[0]
)

ax[0].set_title("SVM Confusion Matrix")
ax[0].set_xlabel("Predicted")
plt.show()

```

OUTPUT:



Result:

The alternative implementations of **Support Vector Machine (SVM)** and **Random Forest** were successfully executed on the Iris dataset. Both models produced accurate predictions, and their performance was analyzed using accuracy scores, classification reports, and confusion matrix visualizations.